

04 — Phases, Requirements & Architecture

Implementation specification for the Lumora control plane (→ Regent AI). Written to be read by a coding agent. Read files 02 and 03 first for constraints. Status: **draft for review**. The four layers are still an open discussion topic — treat layer scope decisions as provisional.

PART I — PHASES

Phase 0 — Foundation (target: 4-6 weeks)

Goal: one agent runs one real content workflow end-to-end, fully instrumented.

Build:

- Control plane skeleton with its own data store and interface boundary
- Agent identity and scope model (layer 3 core — this is the irreversible decision)
- Append-only event log
- Minimal run/step recording (layer 1)
- One Lumora workflow: research → draft → review → publish

Exit criteria:

- Every agent action passes through the policy enforcement point; no bypass path exists
- Every action produces an event with agent identity, scope used, and outcome
- You can answer “which agent did this, using what, and was it allowed?” for any output
- Lumora code contains zero direct reads of control plane tables
- Kill switch works: one command halts a named agent mid-run

Do NOT build in phase 0: dashboards, multi-tenancy, marketplace anything, blockchain anything, a UI beyond a CLI and a single read-only web view.

Phase 1 — Lumora operating (target: 3-6 months)

Goal: Lumora produces content at real volume with real revenue attached, and the control plane is what keeps it from breaking.

Build:

- Agent fleet — multiple specialised agents across the content pipeline
- Layer 1 complete: cost, latency, token, error tracking with budgets
- Layer 2: lineage from artifact back to prompt version, model version, input data
- Layer 3 complete: approval gates, scope grants, revocation, halt
- Library Framework integrated as the content taxonomy
- Publishing integrations and affiliate tracking

Exit criteria:

- Revenue exists
- An agent has misbehaved at least once and you found it from the audit trail alone, without reading application logs
- Cost per content piece is known and attributable per agent
- You can roll back a published artifact and show who authorised the publish

Gate before Phase 2: one external paying customer who is not yours. Not a partner, not a friend, not an in-house company. Someone who can complain, negotiate, and churn.

Phase 2 — Regent AI split (trigger-based, not date-based)

Trigger: a second real user of the control plane exists.

Build:

- Extract control plane into a standalone service with its own repo, deploy pipeline, and versioned public API
- Multi-tenancy: tenant isolation on every table and every API call
- Layer 4: evidence packs — exportable, verifiable bundles for auditors and insurers
- SDK for external integration

Exit criteria:

- Lumora consumes Regent purely through the public API, with no special access
 - A second tenant runs in production
 - An evidence pack has been handed to someone external and accepted
-

Phase 3 — Ecosystem (trigger-based)

NeurX trigger: an outside party wants to sell their agent to other users inside the system.

SentientNet trigger: real money flows between multiple parties in volumes that cannot be split manually.

Neither is built speculatively. If the trigger does not fire, the project does not exist. That is an acceptable outcome.

PART II — THE FOUR LAYERS, IN DETAIL

Layer 1 — Observability

Question answered: which agent did what, when, at what cost.

Captures:

- Run: id, agent identity, trigger source, start/end, status, parent run
- Step: id, run id, sequence, type (llm_call | tool_call | subagent | human_gate), input hash, output hash, duration, error
- Cost: tokens in/out, model, unit price at time of call, computed cost
- Resource: latency percentiles, retry count, rate limit hits

Build note: use OpenTelemetry semantics for traces and spans rather than inventing a schema. Run maps to trace, step maps to span. This means existing tooling works immediately and you are not locked into your own format.

Commercial reality: crowded space — Datadog, LangSmith, Langfuse, Arize. Do not try to win here. Build the minimum needed to operate, and make it exportable.

Layer 2 — Lineage & attribution

Question answered: this output came from what, exactly.

Captures:

- Artifact → run → step chain
- Prompt template id and version hash at execution time
- Model id and version at execution time
- Input data references — which source documents, which taxonomy nodes, which prior artifacts
- Transformation chain when an artifact is derived from another artifact

Why it matters operationally: when quality drops, this is how you find the cause. When an artifact must be recalled, this is how you find everything derived from it.

Why it matters commercially: this is the substrate for layer 4. Evidence packs are lineage plus attestation.

Critical design point: prompts and configuration must be **content-addressed** — store a hash and resolve to the exact text. Never store “prompt v2” as a mutable pointer, or historical lineage becomes false as soon as v2 is edited.

Layer 3 — Policy & permission ← DESIGN THIS FIRST

Question answered: what may this agent do, who says so, and can I stop it.

This is the only layer that cannot be retrofitted. Identity and scope decisions propagate into every table, every API call, and every agent invocation. Getting it wrong means a rebuild. Layers 1, 2 and 4 can always be extended later from data already captured.

Core concepts

Principal — anything that can act. Three kinds:

- `human` — a person
- `agent` — an autonomous software actor
- `service` — a system integration

Agent identity must be:

- Stable across runs (not per-invocation)
- Versioned (agent v3 is distinguishable from agent v2 in the audit trail)
- Owned by a principal (an agent always has a responsible party)
- Revocable (an agent can be disabled instantly and permanently)

Capability — a named, granular permission. Design them as verbs on resources, not roles:

- `content.draft.create`
- `content.publish.execute`
- `external.http.get`
- `credentials.affiliate.read`
- `agent.subagent.invoke`

Avoid coarse roles like “editor” — roles are compositions of capabilities, defined at a layer above.

Grant — a binding of capability to principal, with:

- Scope constraints (which resources, which accounts, which channels)
- Expiry (grants should default to expiring; permanent grants are an explicit exception)
- Delegation depth (can this agent grant this to a subagent? usually no)
- Issuer (who granted it — always a human at the root of the chain)

Policy — a rule evaluated at decision time. Inputs: principal, capability requested, resource, context (time, cost so far, recent error rate). Output: `allow` | `deny` | `require_approval`.

Approval gate — a human decision point. Must record: who approved, when, what they saw at the time, and how long it took.

Halt — the ability to stop a run mid-flight. Must work at three levels: single run, single agent across all runs, entire fleet.

Non-negotiable rules

1. **No bypass path.** Every external effect — an HTTP call, a publish, a credential read, a spend — goes through the policy enforcement point. If a code path can act without a decision, layer 3 is decorative.
2. **Deny by default.** An unrecognised capability request is denied, not allowed.

3. **Humans at the root.** Every grant chain terminates at a human principal. An agent cannot bootstrap its own authority.
4. **Decisions are events.** Every allow and every deny is written to the event log, including the policy version that decided it.
5. **Scope narrows, never widens, on delegation.** A subagent can never receive more than its parent holds.

Recommended implementation

Do not hand-roll a policy language. Use a proven policy engine — Cedar or OPA — with policies stored as versioned files in the repo, loaded at startup, and hashed into every decision event. This gives you testable, reviewable, diffable policy from day one.

Layer 4 — Evidence

Question answered: can I hand this to an auditor or insurer and have them accept it.

Deliverable: an evidence pack — a self-contained bundle covering a defined scope and period, containing:

- The event log segment, hash-chained and verifiable
- Policy versions in force during the period
- Agent inventory with identities, versions, and grants held
- Incident record: denials, halts, approval gate outcomes
- Attestation: a signature over the bundle, with a documented verification method
- A human-readable summary that does not require reading raw logs

Requirements that constrain earlier layers — build these in from phase 0:

- **Append-only event log.** No updates, no deletes. Corrections are new compensating events.
- **Hash chaining.** Each event carries the hash of the prior event, so tampering is detectable.
- **Clock discipline.** Server-assigned timestamps, monotonic sequence numbers per stream. Never trust an agent's own clock.
- **Completeness proof.** The log must be able to demonstrate that no events are missing

— sequence gaps must be detectable.

Market note: independent third-party audit is projected at roughly 37% of the AI assurance market in 2026, driven by board and insurer confidence. Insurance is the demand mechanism: audits give underwriters the quantifiable data needed to price AI exposure at all. Design the evidence pack for an insurance underwriter as the reader, not a compliance officer — it is the more demanding and more valuable audience.

PART III — REQUIREMENTS

Functional requirements

FR-1 Identity

- FR-1.1 The system SHALL assign every agent a stable, versioned identity independent of invocation.
- FR-1.2 Every agent identity SHALL have exactly one owning principal.
- FR-1.3 The system SHALL support instant, irreversible revocation of an agent identity.
- FR-1.4 The system SHALL distinguish agent versions in all audit output.

FR-2 Authorisation

- FR-2.1 Every capability request SHALL be evaluated against policy before the effect occurs.
- FR-2.2 The system SHALL deny any capability not explicitly granted.
- FR-2.3 Grants SHALL support scope constraints, expiry, and delegation depth.
- FR-2.4 Delegated grants SHALL NOT exceed the parent's scope.
- FR-2.5 The system SHALL support `require_approval` as a policy outcome with a blocking human gate.

FR-3 Execution control

- FR-3.1 The system SHALL halt a single run on demand.
- FR-3.2 The system SHALL halt all runs of a named agent on demand.
- FR-3.3 The system SHALL halt the entire fleet on demand.

- FR-3.4 Halt SHALL take effect before the next external effect, not merely at step boundaries.

FR-4 Recording

- FR-4.1 Every run and step SHALL be recorded with agent identity, timing, and outcome.
- FR-4.2 Every policy decision SHALL be recorded, including denials.
- FR-4.3 Cost SHALL be recorded per step and attributable per agent and per artifact.
- FR-4.4 Prompt and configuration versions SHALL be content-addressed.

FR-5 Lineage

- FR-5.1 Every artifact SHALL be traceable to the run and steps that produced it.
- FR-5.2 Derived artifacts SHALL record their source artifacts.
- FR-5.3 The system SHALL answer "what else came from this input?" for any input.

FR-6 Evidence

- FR-6.1 The event log SHALL be append-only and hash-chained.
- FR-6.2 The system SHALL export a scoped, verifiable evidence pack.
- FR-6.3 Sequence gaps in the event log SHALL be detectable.

FR-7 Transparency (compass requirements)

- FR-7.1 A user SHALL be able to export all their data in an open format.
- FR-7.2 The system SHALL be able to explain any decision in human-readable terms, citing the policy that produced it.
- FR-7.3 No capability available to the operator SHALL be structurally unavailable to a tenant.

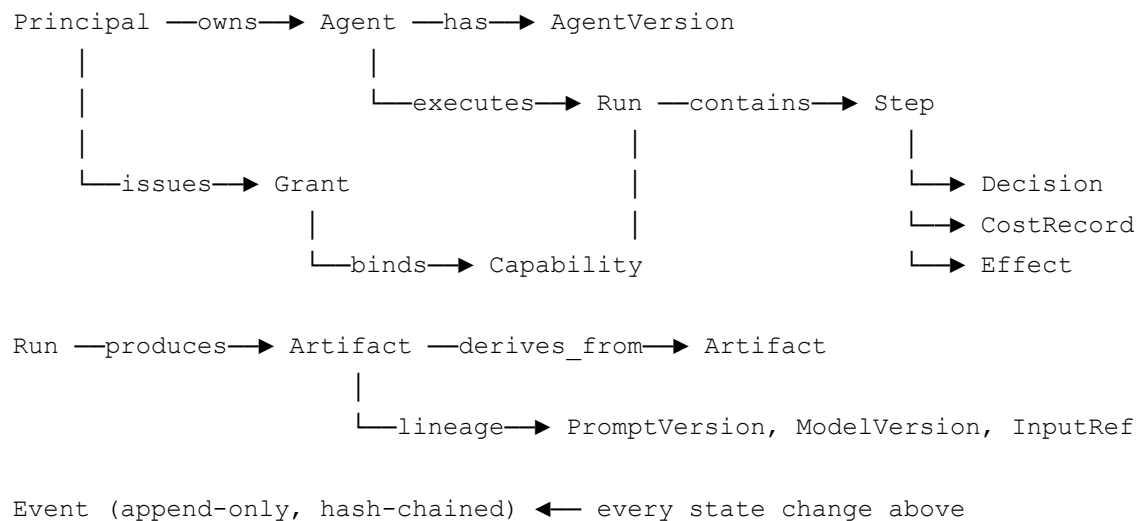
Non-functional requirements

- **NFR-1 Boundary.** The control plane SHALL be a separate module with its own data store. Application code SHALL NOT read control plane tables directly. Violation of this is a build-breaking error, not a style issue.
- **NFR-2 Overhead.** Policy decision latency SHALL be under 10ms at p99 for cached decisions.

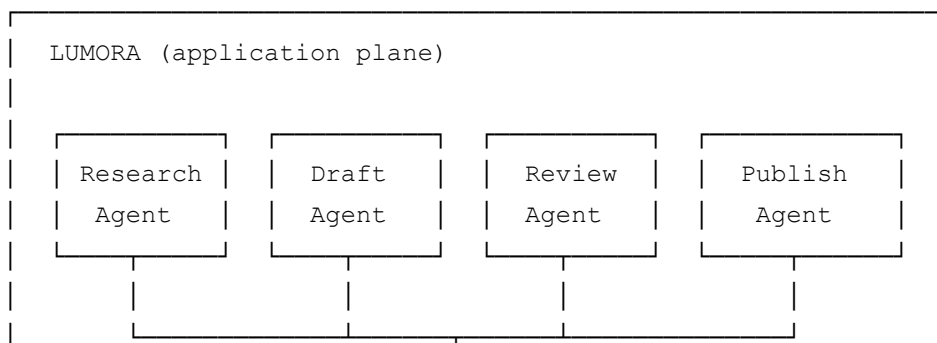
- **NFR-3 Availability posture.** If the control plane is unavailable, agents SHALL fail closed — they stop. They do not proceed unaudited.
- **NFR-4 Durability.** Event log writes SHALL be durable before the corresponding effect executes.
- **NFR-5 Portability.** No dependency on a single cloud provider's proprietary service in the core.
- **NFR-6 Cost.** Phase 0-1 infrastructure SHALL run under a hobby-scale budget — single VM plus managed Postgres is sufficient.

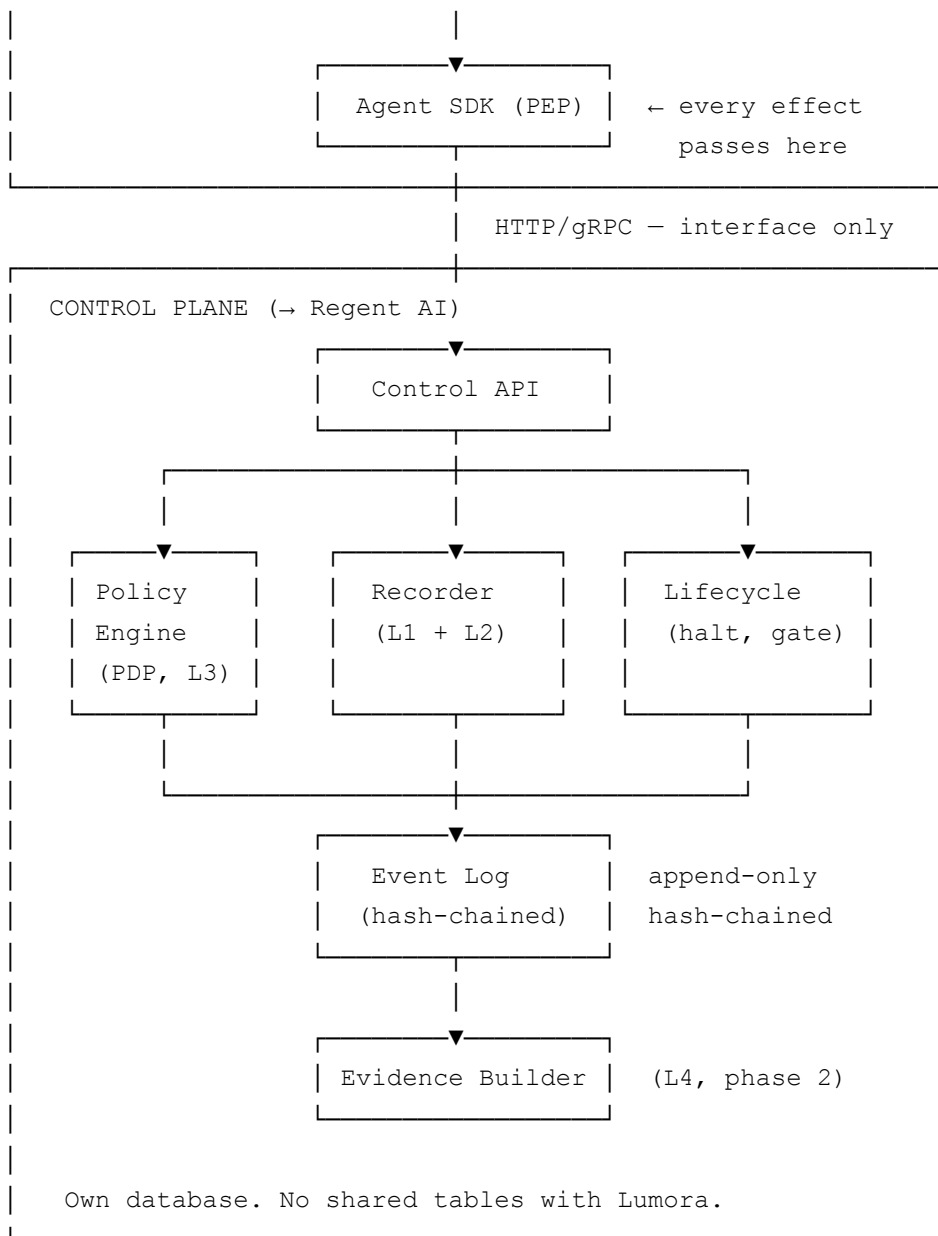
PART IV — ARCHITECTURE

Domain model



Component architecture





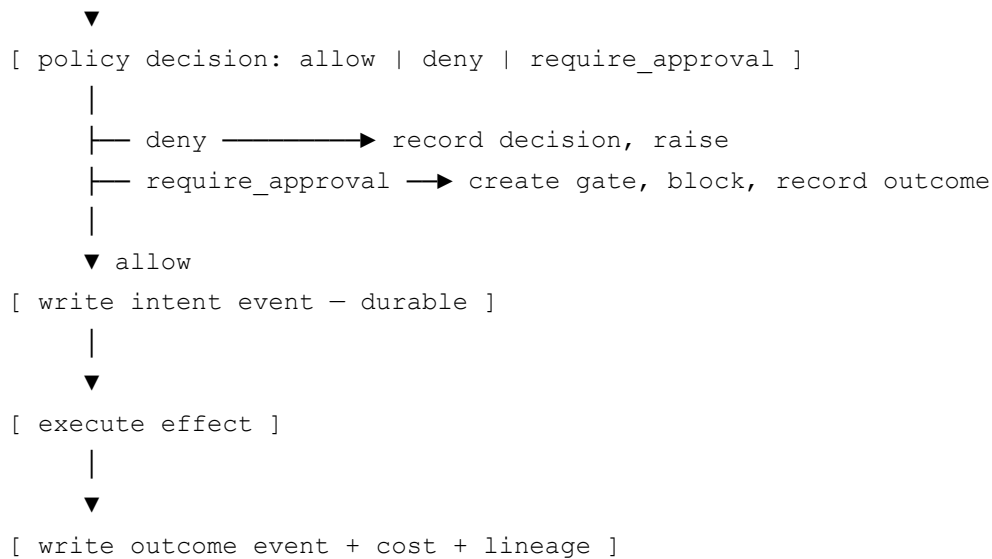
The enforcement point

This is the single most important piece of code in the system. Every agent effect flows through it.

```

effect requested
  |
  ▼
[ resolve principal + agent version ]
  |
  ▼
[ check halt state ] —halted→ abort, record
  |

```



Note the ordering: the intent event is durable *before* the effect executes. This is what makes the audit trail trustworthy — an effect can never exist without a preceding record.

PART V — SOLUTION DESIGN

Main framework

Concern	Choice	Rationale
Language	Python 3.12+	Matches existing skill set; best agent ecosystem
API	FastAPI	Async, typed, OpenAPI generation free
Data	PostgreSQL 16	Event log, relational integrity, JSONB for flexible payloads. One database, separate schemas per module.
Policy	Cedar or OPA	Do not hand-roll. Versioned policy files in repo.
Tracing	OpenTelemetry	Run→trace, step→span. Avoids inventing a schema.
Queue	Postgres-backed job queue initially	Do not add Kafka/Redis until Postgres actually hurts.
Agent runtime	Whatever Lumora uses	The control plane must be runtime-agnostic — this is a design requirement, not an accident.
	Content-addressed	

Config	files, hash in every event	Mutable config pointers make lineage false
--------	----------------------------	--

Repository layout — monorepo with hard boundaries:

```

/lumora          application plane
  /agents
  /workflows
  /library        Library Framework
/controlplane    → becomes Regent AI
  /api
  /policy
  /recorder
  /events
  /evidence
  /migrations
/sdk             the PEP, imported by lumora
/policies        .cedar policy files, versioned
/ops             runbooks, dashboards, alerts
/infra          terraform / compose

```

Boundary enforcement — add to CI on day one:

- An import-linter rule forbidding `lumora/*` from importing `controlplane/*` (only `sdk/*` is allowed)
- A test asserting no Lumora migration touches control plane schemas
- A test asserting every external-effect function is decorated with the enforcement wrapper

Without these, the boundary erodes within weeks and the phase 2 split becomes a rewrite.

Operations framework

Kill switch — the first operational feature, before any dashboard.

```

regent halt --run <id>
regent halt --agent <name>
regent halt --all
regent resume --agent <name> # requires human principal + reason, recorded

```

Budgets — every agent has a cost ceiling per period. Exceeding it is a policy denial, not an alert. Alerts get ignored; denials do not.

SLOs for phase 1 (deliberately modest):

- Policy decision p99 < 10ms cached, < 100ms uncached
- Event write durability 100% — this is the one that cannot slip
- Control plane availability 99% — and agents fail closed when it is down

Runbooks needed before phase 1 ends:

1. Agent produced bad output → find it, halt it, roll back, identify blast radius via lineage
2. Cost spike → identify agent, apply budget denial, investigate
3. Credential leak suspicion → revoke agent identity, enumerate every action taken with it
4. Control plane down → confirm agents halted, restore, verify no unaudited effects occurred

Weekly review ritual (30 minutes, phase 1 onward): denials by agent, cost per artifact, approval gate latency, any event log gaps. This is where you learn what the control plane actually needs — the same learning that becomes Regent's product spec.

Infrastructure preparation

Phase 0-1 — deliberately small:

- Local: docker compose — Postgres, control plane, Lumora
- Deployed: one VM plus managed Postgres. Nothing else.
- Secrets: a managed secret store from day one, never environment files in git
- Backups: automated daily Postgres snapshot, restore tested once before phase 1 ends

Explicitly do NOT provision in phase 0-1: Kubernetes, service mesh, multi-region, Kafka, a data warehouse, a feature store. Each is a month of work that produces no learning.

Phase 2 additions when the split happens:

- Separate deployments for Lumora and Regent
- Tenant isolation verified by test, not by convention
- Object storage for evidence packs, with immutability/retention lock enabled

Deployment framework

- **Migrations:** forward-only, reversible within one version, run before deploy. The event log schema is append-only in structure too — add columns, never repurpose them.
 - **Environments:** local → staging → production. Staging must have a real Postgres, not SQLite.
 - **CI gates:** boundary lint, policy unit tests, migration dry-run, enforcement-wrapper coverage test.
 - **Policy deployment:** policies version independently of code. A policy change is a reviewed, hash-recorded event — never a hot edit.
 - **Rollback:** code rolls back freely. Event log never rolls back — corrections are compensating events.
 - **Feature flags:** for Lumora features only. Never for enforcement — the enforcement point has no off switch, by design.
-

PART VI — IMPLEMENTATION ORDER

Strictly sequential. Each step is unusable without the previous one.

1. Postgres schema: principals, agents, agent_versions, capabilities, grants, events
2. Event log with hash chaining plus a verification function, and its tests
3. Policy engine integration, deny-by-default, with the first three capabilities defined
4. The SDK enforcement wrapper — the piece every effect passes through
5. Halt mechanism at all three levels
6. Run/step recorder (layer 1)
7. First Lumora agent, using the SDK exclusively for every effect
8. Approval gate and its blocking flow
9. Lineage capture on artifact creation (layer 2)
10. Cost recording and budget denial
11. Second, third and fourth Lumora agents
12. Read-only web view: runs, decisions, costs
13. — phase 1 operating —

14. Evidence pack builder (layer 4)
15. Multi-tenancy
16. Public API and extraction into a separate service

Steps 1-5 are the irreversible foundation. Everything after is additive. Take the time on those.

OPEN QUESTIONS FOR THE NEXT DISCUSSION

1. **Capability granularity.** Verb-on-resource is proposed above, but the specific capability list for Lumora's content pipeline is undefined. Too fine and it is unusable; too coarse and layer 3 is decorative.
2. **Approval gate ergonomics.** A blocking human gate in an automated content pipeline is a throughput problem. Which actions genuinely warrant one — publishing? spending? touching credentials?
3. **Scope model for external accounts.** How are per-platform publishing credentials scoped so one compromised agent cannot post everywhere?
4. **Layer 4 target reader.** Insurance underwriter versus compliance officer produces materially different evidence pack design. This choice shapes what layers 1-2 must capture.
5. **Where the taxonomy boundary sits.** Library Framework is application-plane, but lineage needs to reference taxonomy nodes — how does that cross the boundary without coupling?